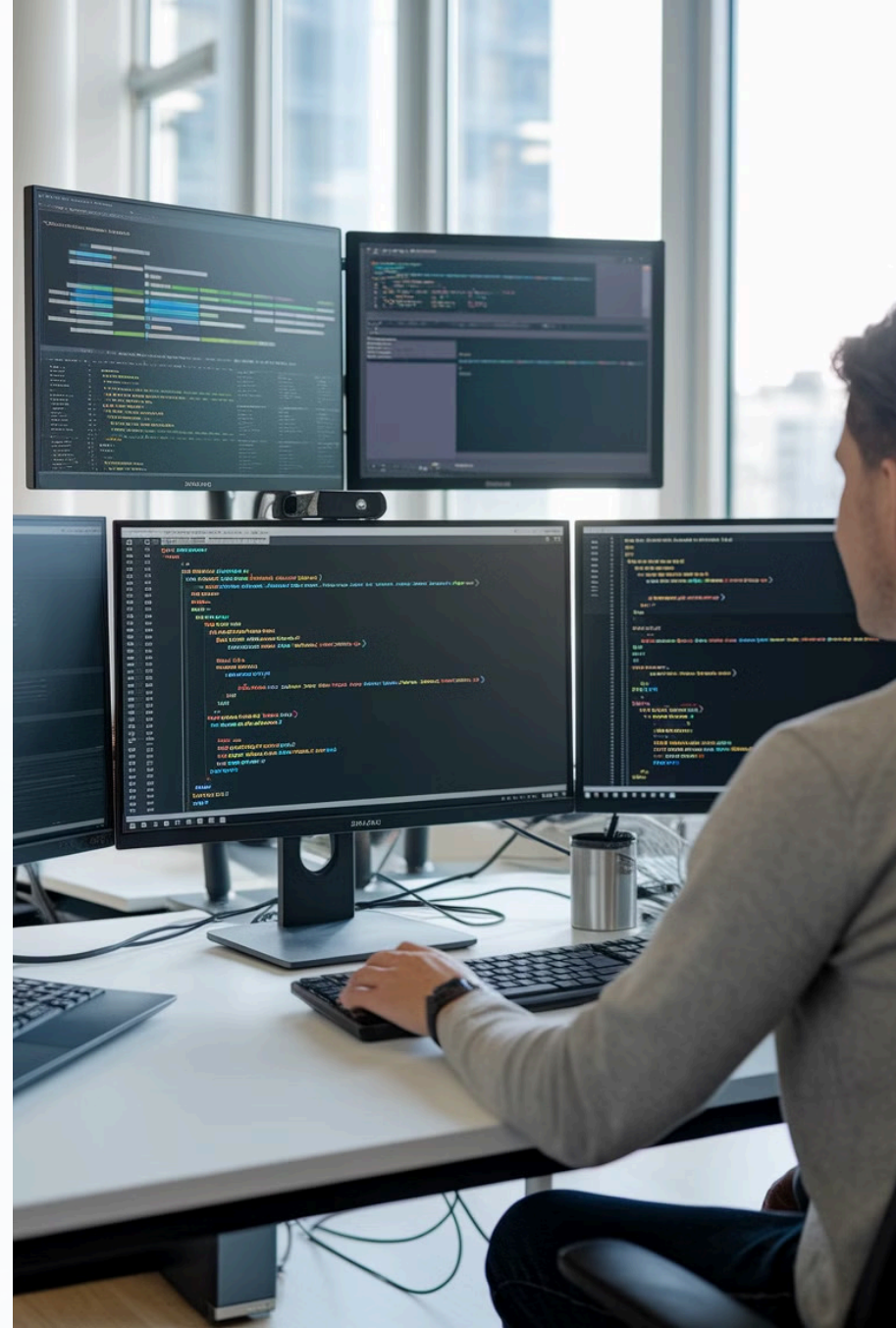


Ferramentas de Build Java: Maven e Gradle nos IDEs Populares

Bem-vindo à nossa apresentação sobre as ferramentas de build Apache Maven e Gradle e como elas são suportadas nos principais ambientes de desenvolvimento Java.

RP by Rodrigo Martins Pagliares





Introdução



Ferramentas de Build

Essenciais para gestão de dependências e automatização do ciclo de vida em projetos Java SE.



Maven e Gradle

As duas soluções mais populares para builds de projetos Java modernos.



Suporte nos IDEs

NetBeans, Eclipse, IntelliJ e VSCode oferecem integrações específicas para estas ferramentas.



Apache Maven: Visão Geral

XML-Based

Utiliza arquivos POM (Project Object Model) em formato XML para configuração do projeto.

Convention over Configuration

Define estruturas padronizadas de diretórios e comportamentos pré-definidos.

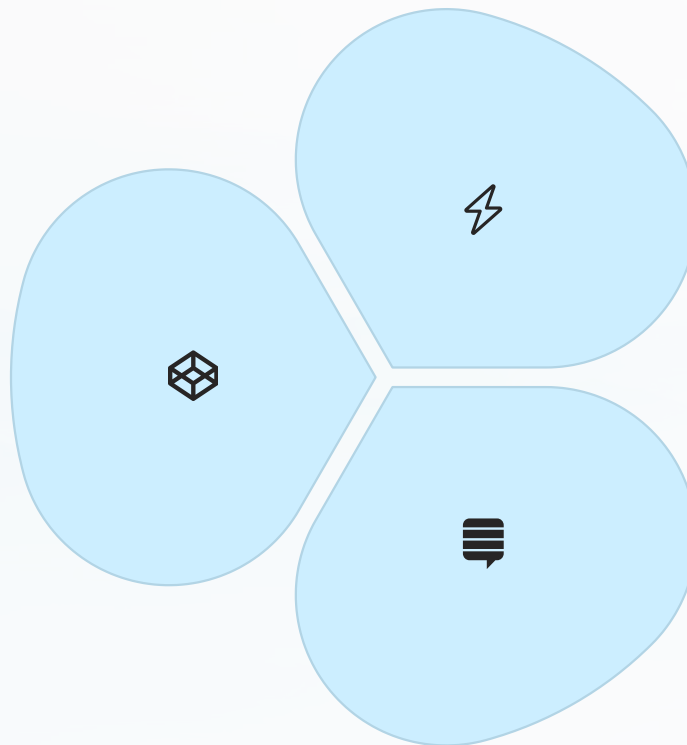
Ciclo de Vida

Possui fases padronizadas de build:
validate, compile, test, package,
verify, install e deploy.

Gradle: Visão Geral

Scripts Flexíveis

Utiliza Groovy ou Kotlin para definição de builds, oferecendo maior programabilidade.



Alto Desempenho

Builds incrementais e execução paralela reduzem significativamente o tempo de compilação.

Compatibilidade Maven

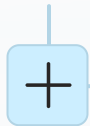
Suporta repositórios Maven existentes e migração facilitada de projetos.



Comparação: Maven vs Gradle

Aspecto	Maven	Gradle
Sintaxe	XML (mais verboso)	Groovy/Kotlin (mais conciso)
Curva de aprendizado	Moderada	Mais acentuada
Velocidade	Mais lento	Significativamente mais rápido
Flexibilidade	Limitada	Alta

NetBeans: Suporte ao Maven



Integração Nativa

Suporte ao Maven embutido sem necessidade de plugins adicionais.



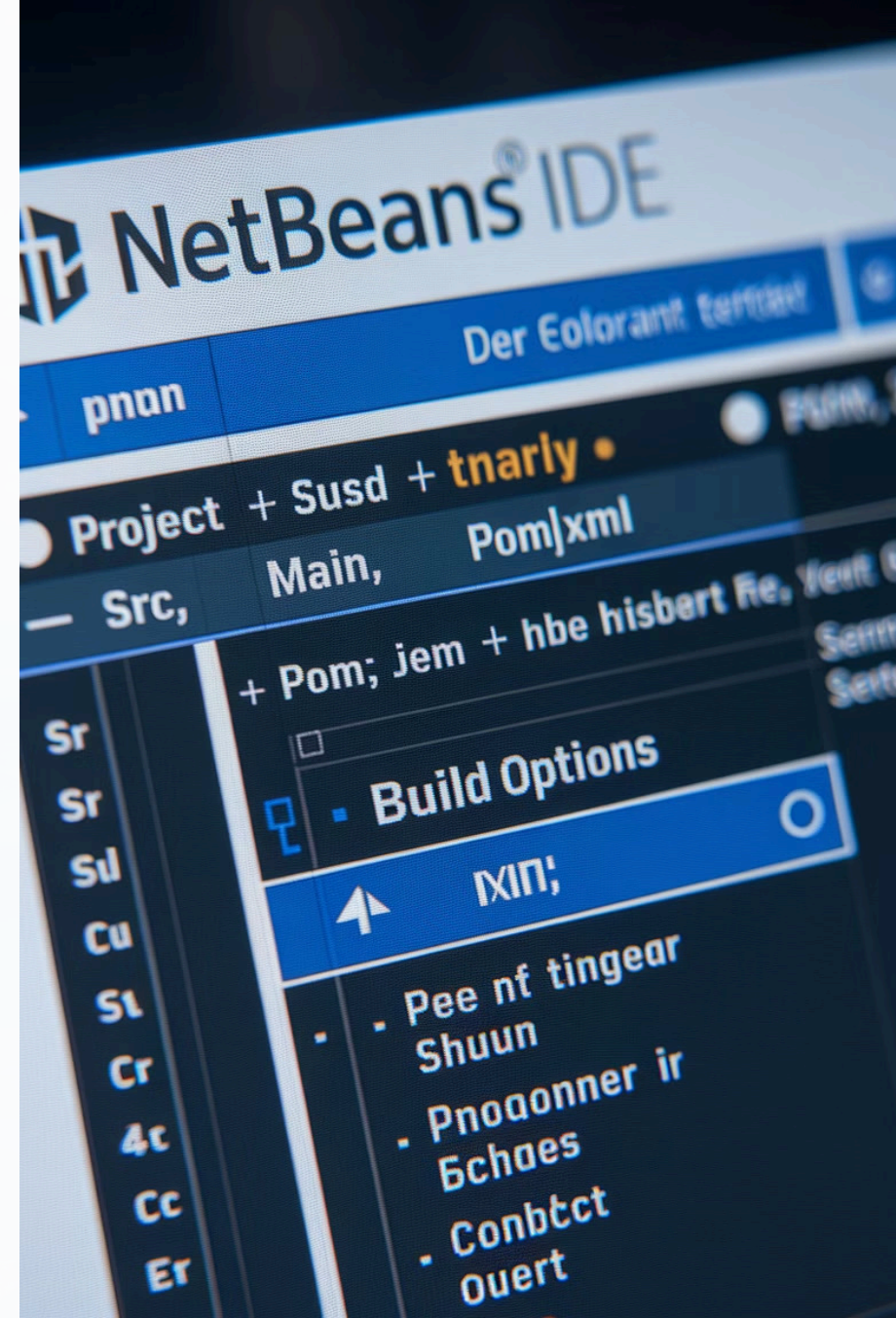
Criação de Projetos

Assistentes dedicados para criação e importação de projetos Maven.



Execução de Goals

Interface gráfica para execução de goals Maven comuns e personalizados.



NetBeans: Suporte ao Gradle



Plugin Gradle

Requer instalação do plugin Gradle para integração completa.



Sincronização

Sincronização automática de mudanças no build.gradle com o IDE.



Execução de Tarefas

Interface para visualizar e executar tarefas Gradle disponíveis no projeto.



Eclipse: Suporte ao Maven

M2Eclipse

Plugin oficial que habilita recursos Maven no Eclipse.

Importação de Projetos

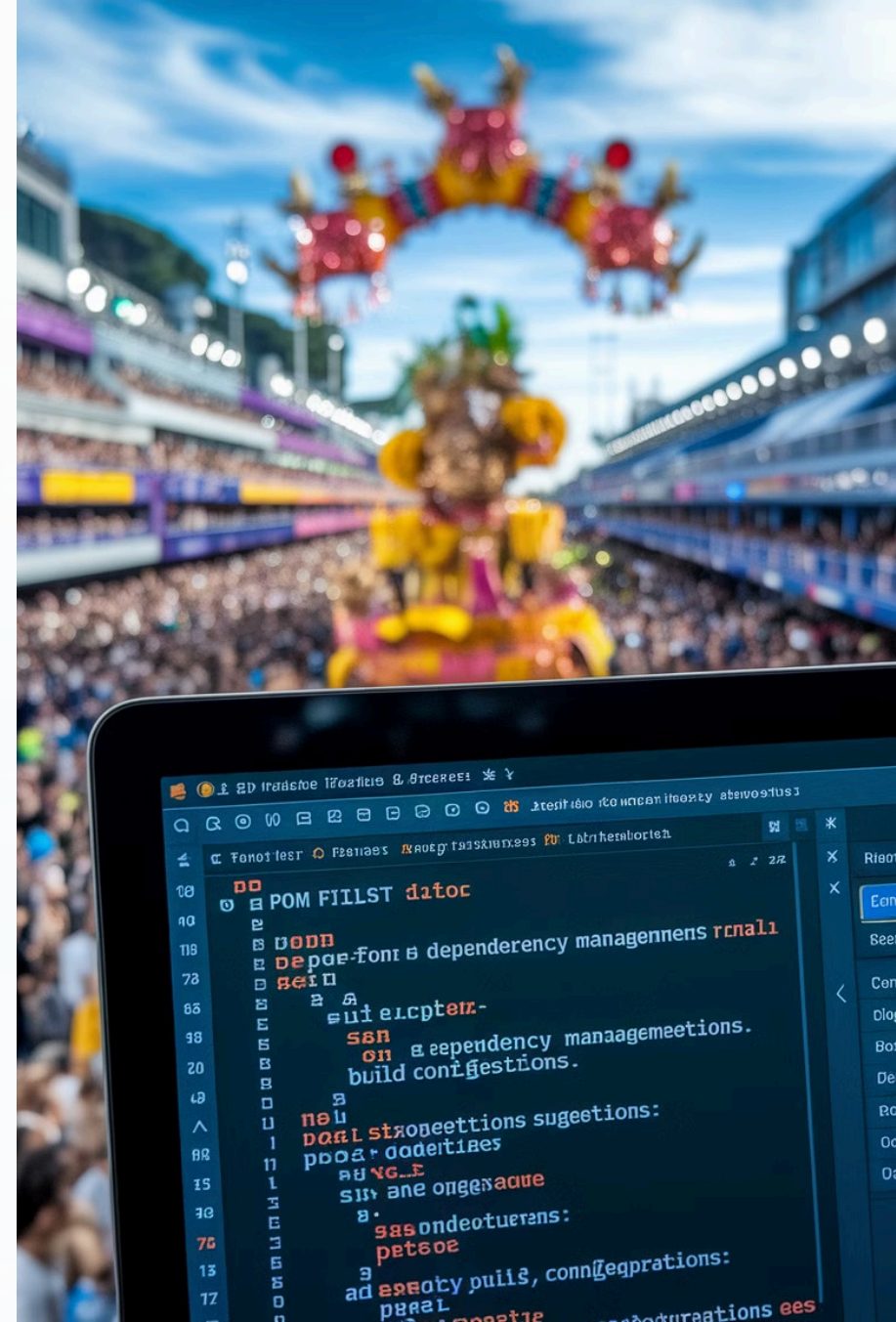
Assistente dedicado para importar projetos Maven existentes.

Gerenciamento de Dependências

Editor visual para manipular dependências no POM.

Run Configurations

Configurações específicas para executar goals Maven.



Eclipse: Suporte ao Gradle



Buildship

Plugin oficial do Gradle para Eclipse que permite integração completa.



Sincronização

Sincronização de modelos Gradle com a estrutura do projeto do Eclipse.

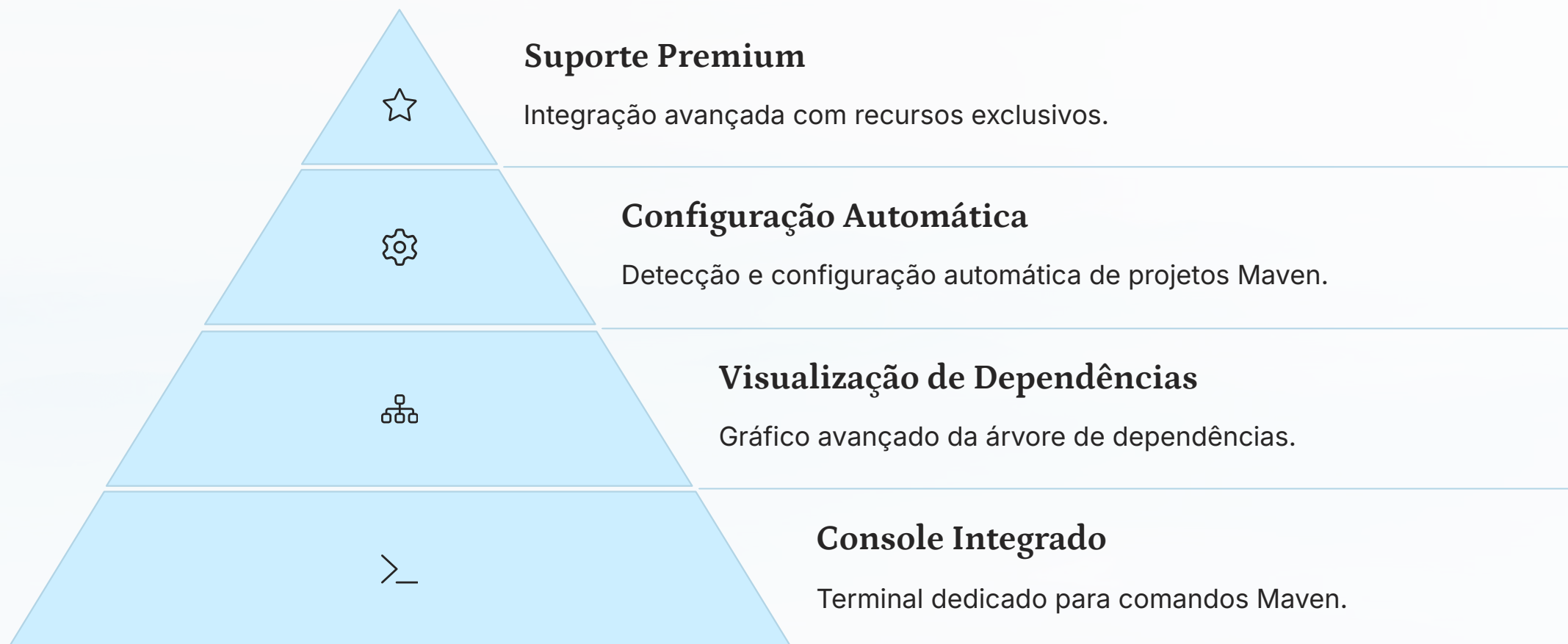


Execução

Painel de tarefas Gradle para execução direta de builds.



IntelliJ IDEA: Suporte ao Maven



IntelliJ IDEA: Suporte ao Gradle



Integração Premium

Suporte nativo de primeira classe ao Gradle.



Editor Inteligente

Autocomplete e validação em tempo real de scripts Gradle.



Explorador de Projetos

Navegação intuitiva pelo modelo de projeto Gradle.

VSCode: Suporte ao Maven



Extensão Maven

A extensão "Maven for Java" adiciona suporte completo no VSCode.

Explorador de Projetos

Visualização de estrutura e dependências de projetos Maven.

Execução Facilitada

Comandos rápidos para executar goals Maven comuns.



VSCode: Suporte ao Gradle

1

Extensão

"Gradle for Java" integra suporte ao Gradle no VSCode.

2

Tarefas

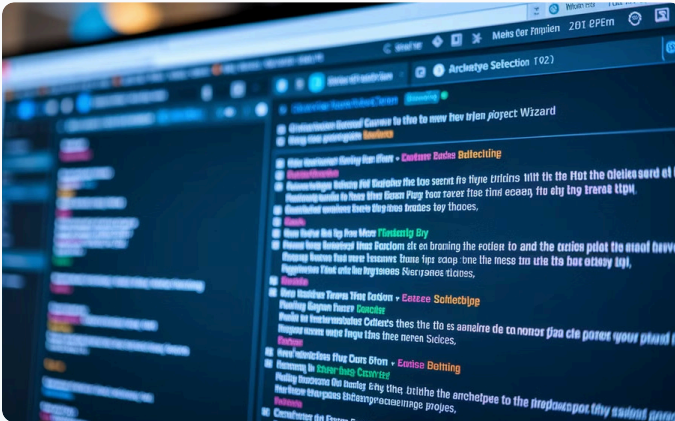
Explorador de tarefas Gradle para visualização e execução.

3

Scripts

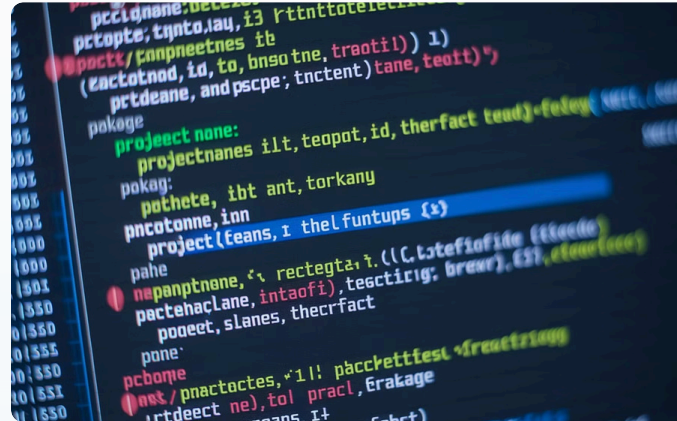
Edição de scripts com sintaxe destacada e validação.

Criação de Projetos: Maven



Eclipse

Criação via wizard M2Eclipse com seleção de archetypes.



NetBeans

Assistente nativo para escolha de archetypes e configurações.



IntelliJ

Diálogo de novo projeto com opções avançadas de configuração Maven.

Criação de Projetos: Gradle



Estrutura

Diretórios `src/main/java`, `src/test/java` e arquivos de configuração Gradle.



`build.gradle`

Arquivo principal com plugins, repositórios e dependências do projeto.



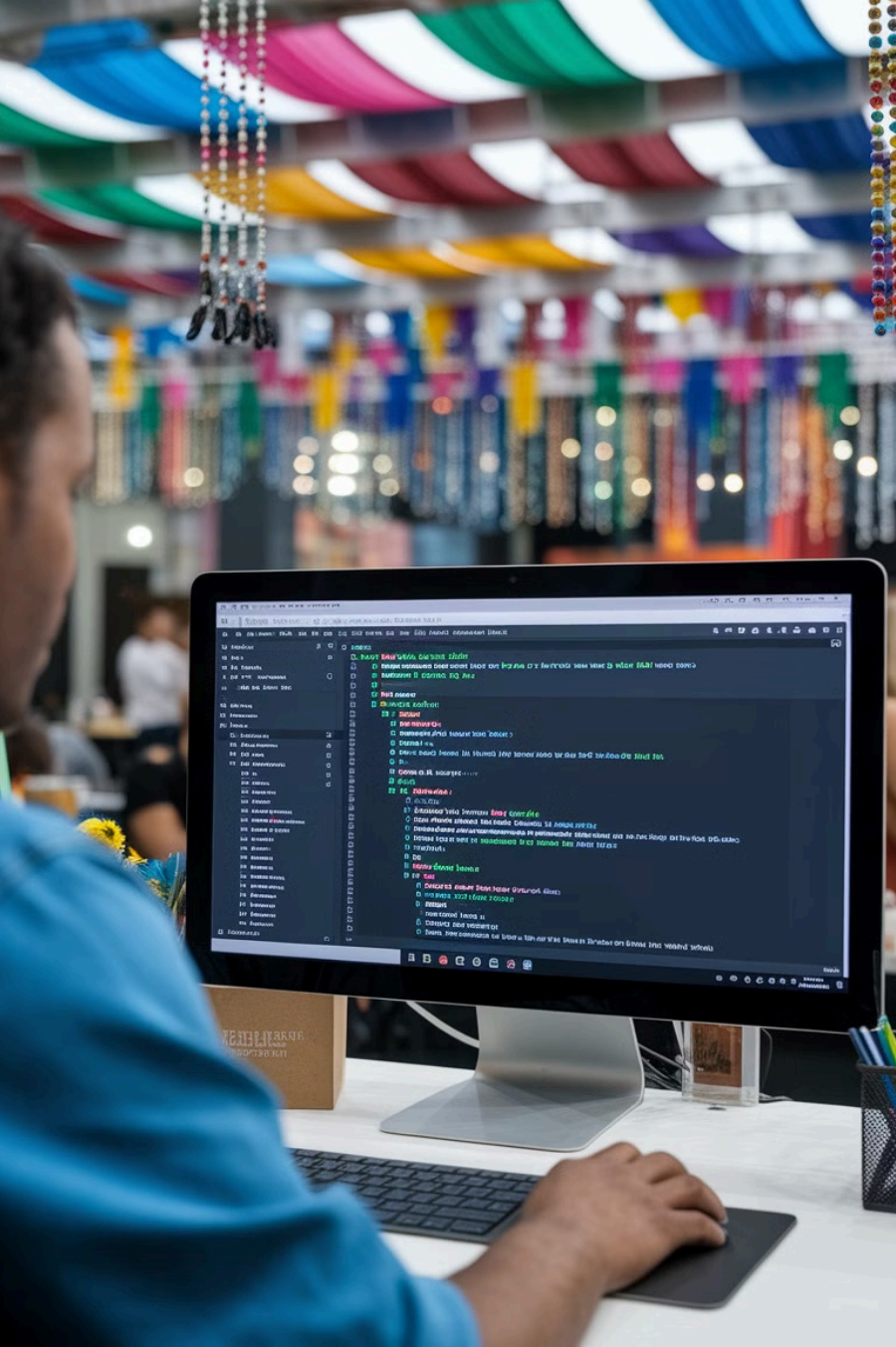
`settings.gradle`

Configura nome do projeto e estrutura multi-módulo.



`gradle/wrapper`

Wrapper para garantir versão consistente do Gradle.



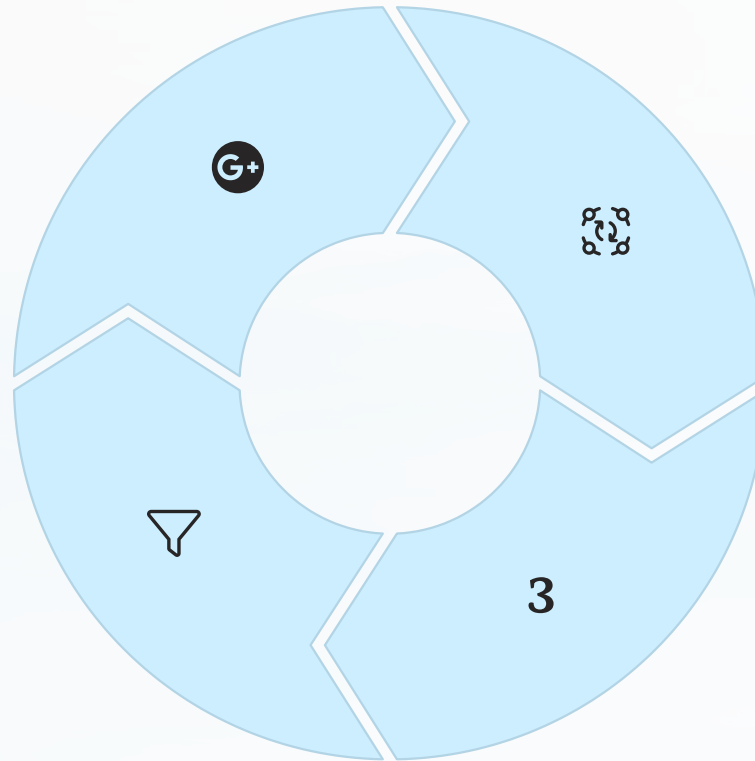
Gerenciamento de Dependências: Maven

Adição

Inserção de dependências no pom.xml com groupId, artifactId e version.

Escopo

Configuração de escopos: compile, test, runtime e provided.



Resolução

IDEs baixam automaticamente dependências dos repositórios configurados.

Visualização

Árvore de dependências mostra relacionamentos e conflitos.

Gerenciamento de Dependências: Gradle

Configuração

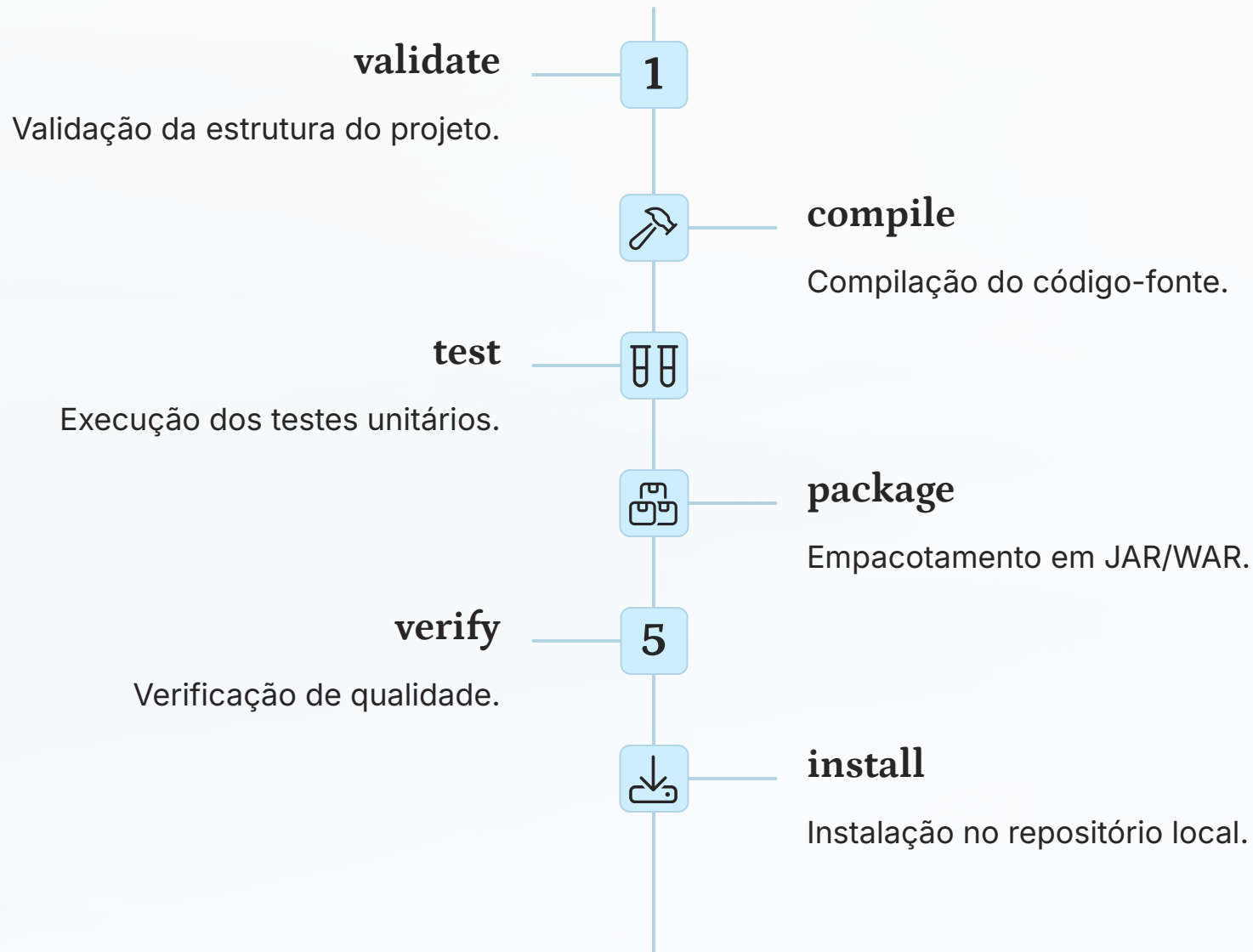
Adição de dependências em blocos no build.gradle:

```
dependencies {  
    implementation 'group:name:version'  
    testImplementation 'junit:junit:4.13'  
    compileOnly 'org.projectlombok:lombok:1.18.20'  
}
```

Vantagens

- Sintaxe mais concisa
- Configuração dinamica
- Resolução avançada de conflitos
- Caching eficiente

Execução de Builds: Maven



Execução de Builds: Gradle



Visualização de Tarefas

Use 'gradle tasks' para ver todas as tarefas disponíveis.



Execução

Execute tarefas específicas como 'gradle build' ou 'gradle test'.



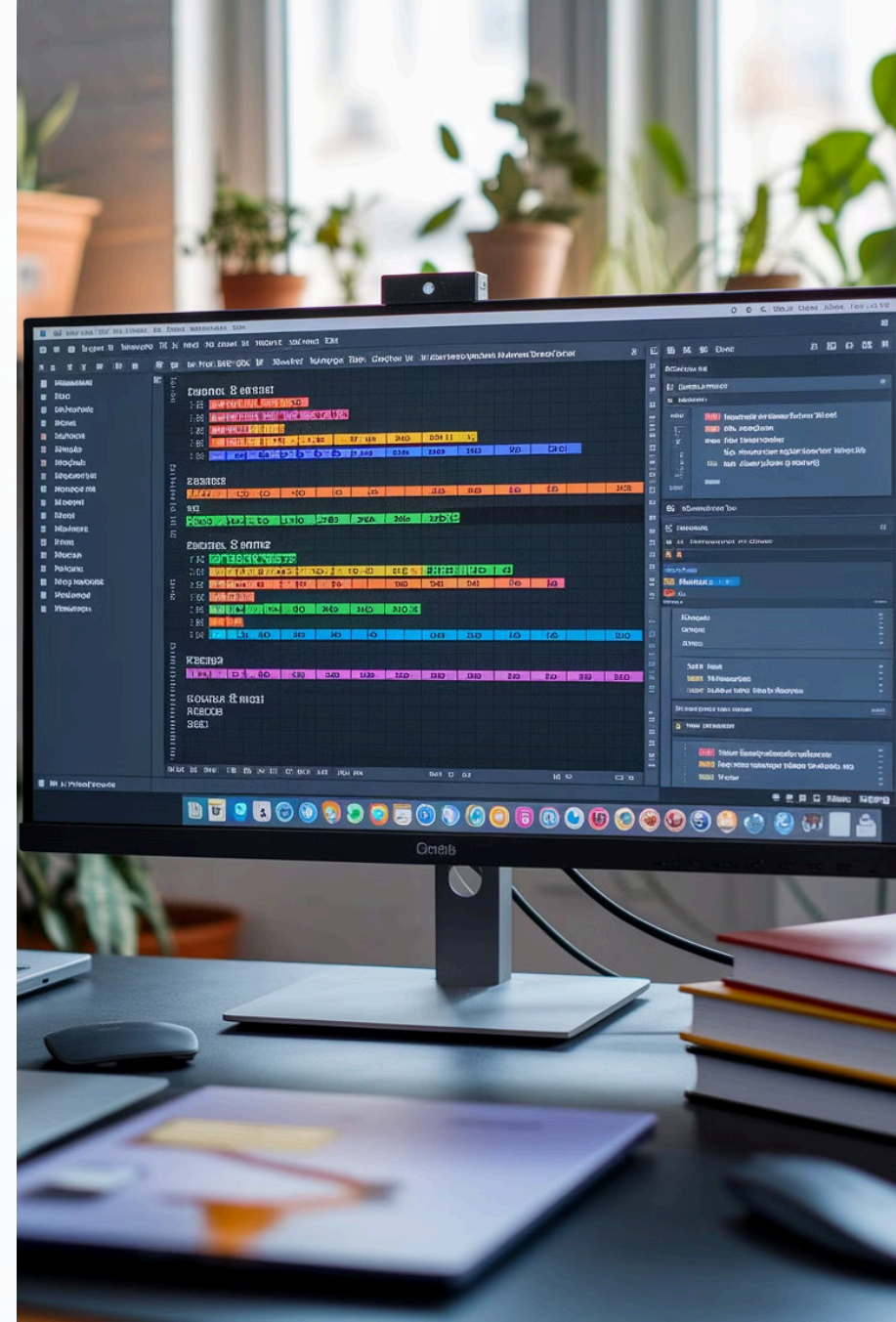
Customização

Crie tarefas personalizadas para fluxos de trabalho específicos.



Paralelismo

Utilize '--parallel' para execução paralela das tarefas.



Depuração e Testes: Maven

Configuração

Plugin Surefire gerencia execução de testes JUnit ou TestNG automaticamente.

Integração com JaCoCo para relatórios de cobertura de código.

Execução nos IDEs

Executar testes diretamente da interface com visualização de resultados.

Depuração passo a passo com pontos de interrupção em código de teste.

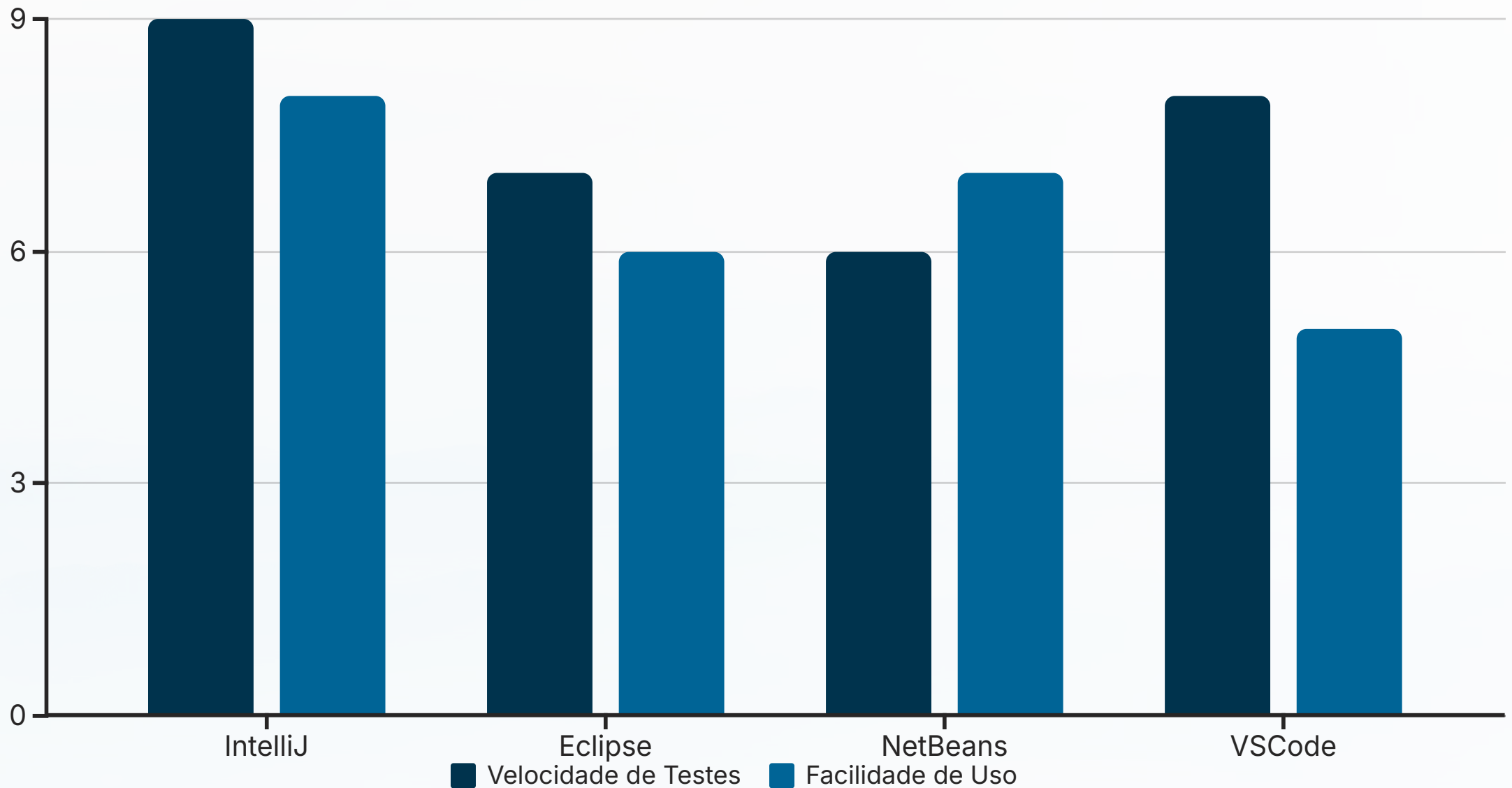
Relatórios

Geração automática de relatórios HTML de resultados de testes.

Visualização gráfica de cobertura de código no projeto.



Depuração e Testes: Gradle



O Gradle oferece execução de testes em paralelo, caching inteligente e relatórios avançados.

O TestKit permite testes automatizados dos próprios scripts de build.



Integração Contínua



Jenkins

Pipelines declarativos com suporte nativo a Maven e Gradle.

- Integração via plugins
- Relatórios de build



GitLab CI

Configuração via `.gitlab-ci.yml` para builds automatizados.

- Runners dedicados
- Caching eficiente



GitHub Actions

Workflows YAML com templates para Java e Maven/Gradle.

- Matrix builds
- Artefatos automatizados

Melhores Práticas

1

Projetos Multi-Módulos

Divida projetos complexos em módulos coesos e bem definidos.

Use parent POMs ou settings.gradle para unificar configurações.

Versionamento

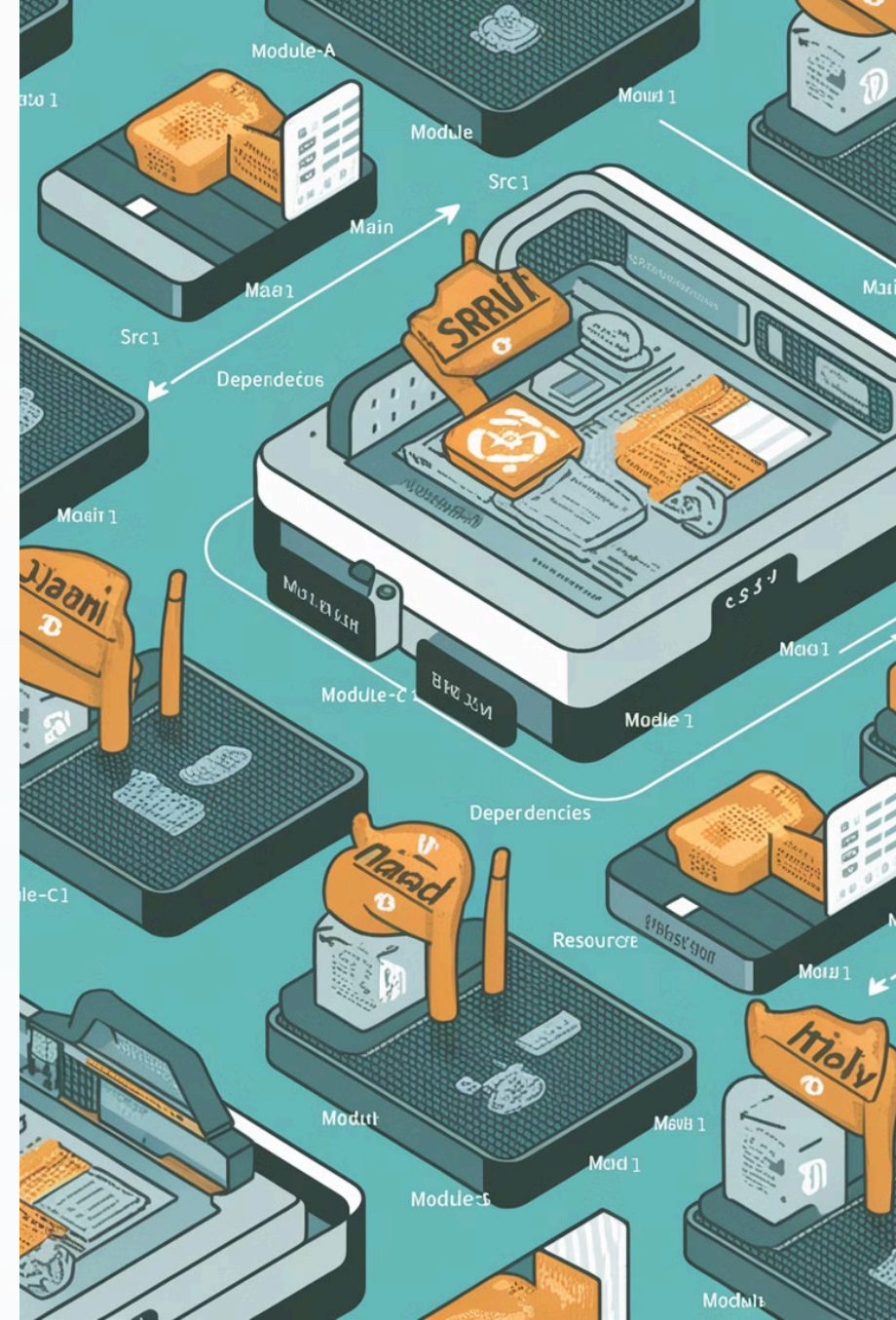
Adote versionamento semântico (MAJOR.MINOR.PATCH).

Utilize BOM (Bill of Materials) para alinhar versões.

Plugins Recomendados

Maven: Compiler, Surefire, Checkstyle, JaCoCo.

Gradle: Java, Application, JaCoCo, Spotless.



Conclusão

Maven

- Mais maduro e estabelecido
- Excelente para projetos simples
- Convenções bem definidas
- Forte ecossistema de plugins

Gradle

- Mais flexível e programável
- Superior em desempenho
- Melhor para projetos complexos
- Curva de aprendizado mais íngreme

IDEs

- IntelliJ: melhor para Gradle
- Eclipse: bom para Maven
- NetBeans: nativo para Maven
- VSCode: leve e flexível



Dúvidas?

Estamos à disposição para esclarecer qualquer dúvida sobre Maven, Gradle ou integração com IDEs Java.

Entre em contato: suporte@javabuildsolutions.com.br